YAŞAR UNIVERSITY

COMP 3328: EMBEDDED SYSTEMS PROJECT

MED-ALERT HEALTH SYSTEM

A SMART EMERGENCY NOTIFICATION SYSTEM

ECEM GÜNEŞ

22070001039

TERM PROJECT ADVISOR: ASSIST.PROF.(PHD) ÖMER ÇETİN

ENGINEERING FACULTY

COMPUTER ENGINEERING PROGRAM

BORNOVA / İZMİR
MAY 2025

APPROVAL PAGE

I acknowledge and declare that I have personally worked in accordance with academic ethical principles during the preparation of this project. I accept and undertake that if the contrary situation is detected in any way, my project work will not be accepted and will not be evaluated, and I will be subject to sanctions announced by the university administration.

Student: **Signature:**

ECEM GÜNEŞ
22070001039
26.05.2025 .....

ABSTRACT

MED-ALERT HEALTH SYSTEM

GÜNEŞ, Ecem

COMP 3328: Embedded Systems Project Term Project,

Advisor: Assist. Prof.(PhD) Ömer ÇETİN

May, 2025

The Med-Alert Health System was developed to support elderly individuals or patients who may experience medical emergencies while alone, especially in home care or assisted living settings. The system allows users to select both the emergency contact (nurse, doctor, or family) and the urgency level (low, medium, high) via a keypad.

Once a selection is made, the system provides immediate feedback using a 16x2 LCD, a 10-segment LED bar graph, and three urgency-level LEDs. For critical alerts, an acoustic buzzer is activated. All events are logged to a microSD card to ensure traceability and post-event analysis.

The system is built on the Arduino Mega microcontroller and programmed in AVR C to enable low-level hardware control. All components are tested using the Wokwi emulator to validate integration and real-time responsiveness.

This project demonstrates how embedded systems can offer a cost-effective and customizable solution for emergency alert needs. By focusing on user-friendliness, reliability, and real-time response, the Med-Alert System contributes to safer, more independent healthcare support for vulnerable individuals.

Keywords: Med-Alert Emergency Notification, Embedded Systems, AVR-based control, Keypad Interface, LCD Display, Buzzer Alert, Wokwi simulation environment

TABLE OF CONTENTS

APPROVAL PAGE iii

ABSTRACT v

TABLE OF CONTENTS vii

1. CHAPTER: PROBLEM DEFINITION 9

2. CHAPTER: THE GOAL OF THE PROJECT 9

3. CHAPTER: HARDWARE 11

3.1. Microprocessor / Microcontroller 11

3.2. Diagram Defination of Project 11

3.3. Other Input / Output Devices Used within the Scope of the Project 12

3.4. Cost of Project 13

4. CHAPTER: SOFTWARE DEVELOPMENT 15

4.1. Functions 15

4.2. Constants & Variables 15

5. CHAPTER: CODE 19

5.2. AVR C Code 19

5.3. WOKWI diagram.json File 26

6. CHAPTER: PROJECT RUNTIME SCREENSHOT IMAGES 29

7. CHAPTER: CONCLUSIONS AND FUTURE WORK 33

8. REFERENCES 33

1. CHAPTER: PROBLEM DEFINITION

In modern healthcare, especially in home care and elderly support, there is an increasing need for simple and accessible emergency alert systems. Many people with chronic illnesses or limited mobility often live alone or with little supervision. In urgent situations like heart attacks or falls, getting help quickly can save lives.

The Med-Alert System was designed to meet this need. It is a small, user-friendly system that allows patients to send alerts using a keypad. The user can choose who to notify (nurse, doctor, or family member) and also select the urgency level (low, medium, or high). This information appears on an LCD screen and is supported by LEDs, a buzzer, and data logging to an SD card.

The system is especially useful in places without constant medical staff, like rural homes, assisted living centers, or even hospital beds with limited monitoring. Med-Alert is more affordable and easier to use than hospital-grade systems. It provides both real-time alerts and records of previous alerts for caregivers.

2. CHAPTER: THE GOAL OF THE PROJECT

The primary objective of the Med-Alert Health System is to develop a responsive and customizable emergency notification platform through an embedded system using the AVR C programming language. The system enables individuals in medical distress to seek assistance quickly and efficiently by selecting both the emergency contact and the level of urgency via a keypad interface.

This manually operated design is especially suitable for semi-autonomous users in home care or assisted living settings. Individuals who experience early warning signs such as chest pain, dizziness, or anxiety can initiate an alert manually, which is displayed on a 16x2 LCD screen and logged to an SD card for future reference. The notification is further reinforced by a visual LED bar graph and three individual LEDs (low, medium, high) to represent the urgency level, along with an audible buzzer that activates in critical conditions.

The project focuses on usability, reliability, and real-time responsiveness, especially for the elderly and those with chronic illnesses. Its intuitive design minimizes user learning curves while providing caregivers with clear and traceable feedback. The system is developed and tested within the Wokwi emulator environment and programmed using AVR C, allowing full control over hardware-level operations including direct manipulation of I/O ports.

Ultimately, the Med-Alert System seeks to bridge the gap between passive health monitoring and active emergency response solutions, empowering users with a simple yet effective way to communicate urgent health needs to caregivers or family members.

3. CHAPTER: HARDWARE

3.1. Microprocessor / Microcontroller

The Arduino Mega microcontroller was chosen for this project because it has many digital and analog I/O pins, which are needed to connect components like the LCD, keypad, buzzer, and LED bar graph. Its support for AVR C programming and SPI communication also makes it suitable for embedded system development and testing in the Wokwi emulator.

1.2. Diagram Definition of Project

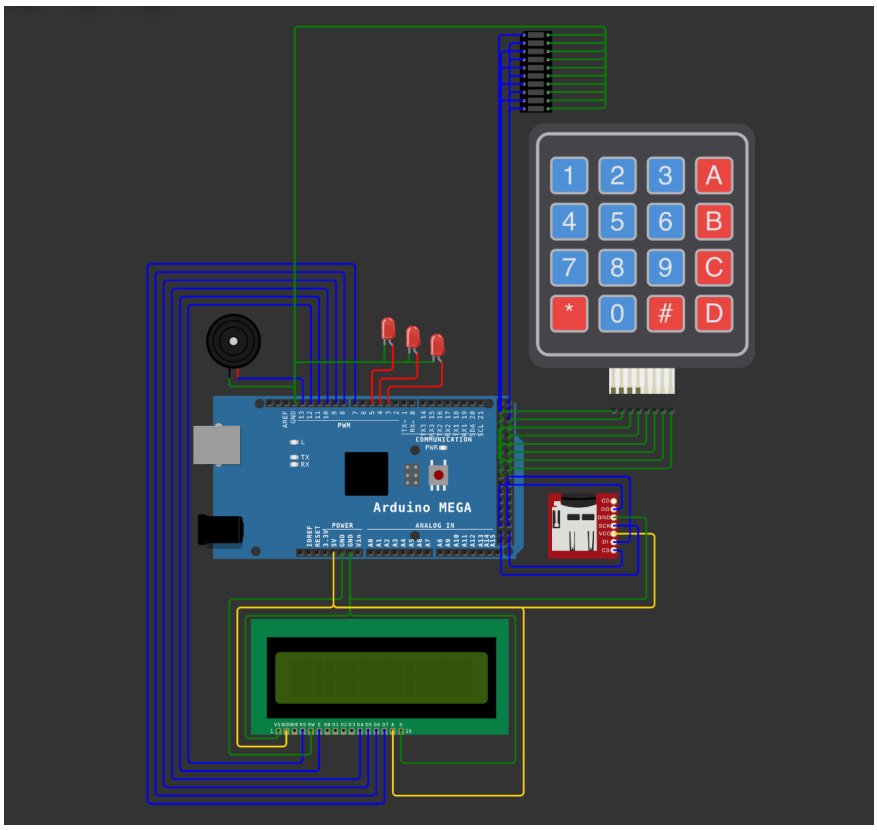


Figure 1.1. Circuit Diagram of Med-Alert System

The diagram above shows the full hardware setup of the Med-Alert Health System using the Arduino Mega. The key input/output

components are connected to specific digital and analog pins on the board.

- **LCD Display 16x2**: Used to show the selected recipient, urgency level, and confirmation messages.
- **Keypad:** Used by the user to select recipient and urgency level via numeric keys.
- **LED Bar Graph:** Indicates the urgency level visually (3, 6, or 9 LEDs).
- **Three separate LEDs**: Used to signal Low, Medium, and High urgency levels individually.
- **Buzzer**: Activated when the Nurse is selected with a Medium or High urgency level.
- **MicroSD Card Module**: Logs the selected recipient and urgency data for record keeping.

1.3. Other Input / Output Devices Used within the Scope of the Project

Type	I/O Device	Purpose of usage
Display	*LED/ RGB LED*	It is planned to be used to visually indicate the selected urgency level (Low, Med, High).
Display	*LCD (16x2 veya 20x4)*	It is planned to be used to display the selected recipient, urgency level, and alert confirmation.
Display	*LED Bar Graph*	It is planned to be used to provide a segmented visual scale (3–9 LEDs) of the urgency level.
Input	*Keypad*	It is planned to be used to allow the user to select a recipient and urgency level via keypad input.
Output	*Buzzer*	It is planned to be used to provide an audible alert when Nurse is notified with Medium or High urgency.
Misc	*MicroSD Card Reader*	It is planned to be used to log notification events into external storage for later review.

1.4. Cost of Project

Component	Estimated Cost
Arduino Mega	$35.00
LCD (16x2)	$9.99
4x4 Keypad	$10.99
3x LED	$1.50
Buzzer	$1.50
LED Bar Graph	$3.00
MicroSD Card Module	$4.50
Jumper Wires	$3.99
TOTAL	**$70.47**

.

4. CHAPTER: SOFTWARE DEVELOPMENT

The software development phase of the Med-Alert Health System was structured around real-time input handling, system status visualization, and alert generation using AVR C programming. The development process did not rely on machine learning or database-based analysis. Instead, the system focused on microcontroller-level real-time decision-making through embedded C functions.

In the Data Description stage, the system was defined to include keypad input processing, emergency level interpretation, LED signaling, buzzer activation, and data logging via SPI communication to a microSD card. Each function was designed and tested in isolation to ensure modularity and ease of debugging.

In the Data Analysis phase, no statistical dataset was used; however, hardware and software testing was carried out within the WOKWI simulation environment. Scenarios such as key press combinations, urgency level transitions, and alert triggering were evaluated through iterative testing to validate system behavior. The final implementation combined all subsystems into a single real-time loop structure suitable for continuous monitoring.

This phase lays the foundation for a functional prototype that can respond to critical medical events and record relevant data for further evaluation.

4.1. Functions

- **lcd_init()**: Initializes the LCD in 4-bit mode and sets cursor and display properties.
- **lcd_command()**: Sends a command instruction to the LCD such as clear screen or set cursor.

- **lcd_data()**: Sends a single character (data) to be displayed on the LCD.
- **lcd_print()**: Sends a full string to be printed on the LCD.
- **lcd_clear()**: Clears all characters on the LCD and resets the cursor.
- **lcd_send_nibble()**: Sends a 4-bit data nibble to the LCD via control pins.
- **lcd_enable()**: Generates a short enable pulse required to latch LCD instructions.
-
- **keypad_init()**: Sets up keypad rows as outputs and columns as inputs with pull-up resistors.
- **keypad_read()**: Scans the keypad matrix to detect which key has been pressed.
-
- **buzzer_init()**: Configures the buzzer pin as output and initializes it in LOW state.
- **buzzer_on()**: Turns on the buzzer by setting the pin high.
- **buzzer_off()**: Turns off the buzzer by clearing the buzzer pin.
- **buzzer_beep_ms(duration)**: Activates the buzzer for a specific time in milliseconds.
-
- **ledbar_init()**: Sets up all 10 pins connected to the LED bar graph as outputs.
- **ledbar_clear()**: Turns off all LEDs in the LED bar graph.
- **ledbar_set_level(level)**: Turns on a defined number of LEDs (3, 6, or 9) according to urgency.
-
- **urgency_leds_init()**: Initializes the three separate LEDs for urgency indication.
- **urgency_leds_clear()**: Turns off all urgency LEDs.
- **urgency_leds_set(count)**: Turns on 1 to 3 LEDs depending on the urgency level.
-
- **spi_init()**: Initializes SPI by setting appropriate control bits and clock rate.
- **spi_transfer(data)**: Sends and receives a byte over SPI.

- **sd_send_command(cmd, arg, crc)**: Sends a command packet to the SD card via SPI.
- **sd_init()**: Initializes the SD card and prepares it for data writing.
- **sd_write_block(text)**: Writes a 512-byte data block to the SD card (raw format).
- **get_log_message(r, u)**: Formats recipient and urgency values into a single log message.

4.2. Constants & Variables

Constants:

- **F_CPU (16000000UL)**: Defines the system clock frequency as 16 MHz for accurate timing.
- **LCD_RS, LCD_E, LCD_D4–D7**: Pin assignments used to control the LCD in 4-bit mode.
- **CS_LOW(), CS_HIGH()**: Macros for controlling SPI chip select pin.
- **row_pins[4], col_pins[4]**: Arrays defining keypad matrix pin assignments.

Variables:

- **char recipient**: Stores which recipient was selected via keypad (‘1’, ‘2’, ‘3’).
- **char k**: Holds the latest key pressed, especially for urgency level selection.
- *char log**: Pointer to the formatted log message before SD writing.
- **uint8_t sd_ready**: Stores the SD card initialization status (1 = ready, 0 = fail).
- **char msg[64]**: Holds the emergency message string generated by get_log_message().

1

5. CHAPTER: CODE

3.2. AVR C Code

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <string.h>
#include <stdio.h>

#define LCD_RS PB6
#define LCD_E  PB5
#define LCD_D4 PB4
#define LCD_D5 PH6
#define LCD_D6 PH5
#define LCD_D7 PH4
#define CS_LOW()  (PORTB &= ~(1 << PB0))
#define CS_HIGH() (PORTB |= (1 << PB0))

uint8_t row_pins[4] = {PC5, PC4, PC3, PC2};
uint8_t col_pins[4] = {PC1, PC0, PD7, PG2};

void lcd_enable() {
  PORTB |= (1 << LCD_E);
  _delay_us(1);
  PORTB &= ~(1 << LCD_E);
```

```
23.   _delay_us(100);
24. }
25.
26. void lcd_send_nibble(uint8_t nibble) {
27.   (nibble & 0x01) ? (PORTB |= (1 << LCD_D4)) : (PORTB &= ~(1 << LCD_D4));
28.   (nibble & 0x02) ? (PORTH |= (1 << LCD_D5)) : (PORTH &= ~(1 << LCD_D5));
29.   (nibble & 0x04) ? (PORTH |= (1 << LCD_D6)) : (PORTH &= ~(1 << LCD_D6));
30.   (nibble & 0x08) ? (PORTH |= (1 << LCD_D7)) : (PORTH &= ~(1 << LCD_D7));
31.   lcd_enable();
32. }
33.
34. void lcd_command(uint8_t cmd) {
35.   PORTB &= ~(1 << LCD_RS);
36.   lcd_send_nibble(cmd >> 4);
37.   lcd_send_nibble(cmd & 0x0F);
38.   _delay_ms(2);
39. }
40.
41. void lcd_data(uint8_t data) {
42.   PORTB |= (1 << LCD_RS);
43.   lcd_send_nibble(data >> 4);
44.   lcd_send_nibble(data & 0x0F);
45.   _delay_ms(2);
46. }
47.
48. void lcd_init() {
49.   DDRB |= (1 << LCD_RS) | (1 << LCD_E) | (1 << LCD_D4);
50.   DDRH |= (1 << LCD_D5) | (1 << LCD_D6) | (1 << LCD_D7);
```

```
  _delay_ms(20);
  lcd_command(0x33); lcd_command(0x32);
  lcd_command(0x28); lcd_command(0x0C);
  lcd_command(0x06); lcd_command(0x01);
  _delay_ms(2);
}

void lcd_clear() { lcd_command(0x01); _delay_ms(2); }
void lcd_print(const char* str) { while (*str) lcd_data(*str++); }

// --- KEYPAD ---
void keypad_init() {
  for (int i = 0; i < 4; i++) { DDRC |= (1 << row_pins[i]); PORTC |= (1 << row_pins[i]);
}
  DDRC &= ~((1 << PC1) | (1 << PC0)); PORTC |= (1 << PC1) | (1 << PC0);
  DDRD &= ~(1 << PD7); PORTD |= (1 << PD7);
  DDRG &= ~(1 << PG2); PORTG |= (1 << PG2);
}

char keypad_read() {
  const char keys[4][4] = {{'1','2','3','A'},{'4','5','6','B'},{'7','8','9','C'},{'*','0','#','D'}};
  for (int r = 0; r < 4; r++) {
    for (int i = 0; i < 4; i++) PORTC |= (1 << row_pins[i]);
    PORTC &= ~(1 << row_pins[r]);
    _delay_us(5);
    for (int c = 0; c < 4; c++) {
      uint8_t val = (c == 0) ? PINC & (1 << PC1) :
                    (c == 1) ? PINC & (1 << PC0) :
```

```
                (c == 2) ? PIND & (1 << PD7) :
                        PING & (1 << PG2);
    if (!val) { _delay_ms(200); return keys[r][c]; }
  }
 }
 return 0;
}

void buzzer_init() { DDRB |= (1 << PB7); PORTB &= ~(1 << PB7); }
void buzzer_on()   { PORTB |= (1 << PB7); }
void buzzer_off()  { PORTB &= ~(1 << PB7); }
void buzzer_beep_ms(uint16_t d) { buzzer_on(); _delay_ms(d); buzzer_off(); }

void ledbar_init() { DDRA = 0xFF; DDRC |= (1 << PC6) | (1 << PC7); }
void ledbar_clear(){ PORTA = 0x00; PORTC &= ~((1 << PC6) | (1 << PC7)); }
void ledbar_set_level(uint8_t l){ ledbar_clear(); for (uint8_t i = 0; i < l; i++) i<8 ? (PORTA |= (1 << i)) : (PORTC |= (1 << PC7)); }

void urgency_leds_init() { DDRE |= (1 << PE3) | (1 << PE5); DDRG |= (1 << PG5); }
void urgency_leds_clear(){ PORTE &= ~((1 << PE3) | (1 << PE5)); PORTG &= ~(1 << PG5); }
void urgency_leds_set(uint8_t c){ urgency_leds_clear(); if (c>=1) PORTE|=(1<<PE3); if(c>=2) PORTE|=(1<<PE5); if(c==3) PORTG|=(1<<PG5); }

void spi_init() { DDRB |= (1 << PB0)|(1 << PB1)|(1 << PB2); DDRB &= ~(1 << PB3); PORTB |= (1 << PB3); SPCR = (1<<SPE)|(1<<MSTR)|(1<<SPR0); }
uint8_t spi_transfer(uint8_t d){ SPDR = d; while (!(SPSR & (1 << SPIF))); return SPDR; }
```

```
uint8_t sd_send_command(uint8_t cmd, uint32_t arg, uint8_t crc){
  CS_LOW(); spi_transfer(0x40 | cmd);
  spi_transfer(arg >> 24); spi_transfer(arg >> 16); spi_transfer(arg >> 8); spi_transfer(arg); spi_transfer(crc);
  for (uint8_t i=0; i<10; i++) { uint8_t r = spi_transfer(0xFF); if (r != 0xFF) return r; } return 0xFF;
}

uint8_t sd_init() {
  CS_HIGH(); for (uint8_t i=0; i<10; i++) spi_transfer(0xFF);
  if (sd_send_command(0, 0, 0x95) != 0x01) return 0;
  if (sd_send_command(1, 0, 0xFF) != 0x00) return 0;
  CS_HIGH(); spi_transfer(0xFF); return 1;
}

uint8_t sd_write_block(const char* txt) {
  CS_LOW();
  if (sd_send_command(24, 0x00000100, 0xFF) != 0x00) { CS_HIGH(); return 0; }
  spi_transfer(0xFE);
  for (uint16_t i=0; i<512; i++) spi_transfer(i<strlen(txt) ? txt[i] : 0x00);
  spi_transfer(0xFF); spi_transfer(0xFF);
  if ((spi_transfer(0xFF) & 0x1F) != 0x05) { CS_HIGH(); return 0; }
  while (!spi_transfer(0xFF));
  CS_HIGH(); spi_transfer(0xFF); return 1;
}

char* get_log_message(char r, char u){
```

```
  static char msg[64];
  const char* rec = (r=='1')?"Nurse":(r=='2')?"Doctor":"Family";
  const char* urg = (u=='4')?"Low":(u=='5')?"Med":"High";
  sprintf(msg, "To:%s, Level:%s", rec, urg);
  return msg;
}

int main(void) {
  lcd_init(); keypad_init(); ledbar_init(); urgency_leds_init(); buzzer_init(); spi_init();
  uint8_t sd_ready = sd_init();

  lcd_print("Select Recipient");
  char recipient = 0;
  while (!recipient) {
    char k = keypad_read();
    if (k=='1'||k=='2'||k=='3') {
      recipient = k; lcd_clear();
      if (k=='1') lcd_print("To: Nurse");
      else if (k=='2') lcd_print("To: Doctor");
      else lcd_print("To: Family");
      lcd_command(0xC0); lcd_print("Level: ");
    }
  }

  while (1) {
    char k = keypad_read();
    if (k=='4'||k=='5'||k=='6') {
      lcd_clear();
```

```
        if (recipient=='1') lcd_print("To: Nurse");
        else if (recipient=='2') lcd_print("To: Doctor");
        else lcd_print("To: Family");

        lcd_command(0xC0); lcd_print("Level: ");
        if (k=='4') { lcd_print("Low"); ledbar_set_level(3); urgency_leds_set(1); }
        else if (k=='5') { lcd_print("Med"); ledbar_set_level(6); urgency_leds_set(2); if(recipient=='1') buzzer_beep_ms(1000); }
        else if (k=='6') { lcd_print("High"); ledbar_set_level(9); urgency_leds_set(3); if(recipient=='1') buzzer_beep_ms(2000); }

        char* log = get_log_message(recipient, k);
        lcd_clear(); lcd_print("Sent to ");
        if (recipient=='1') lcd_print("Nurse");
        else if (recipient=='2') lcd_print("Doctor");
        else lcd_print("Family");
        lcd_command(0xC0);
        if (k=='4') lcd_print("Level: Low");
        else if (k=='5') lcd_print("Level: Med");
        else if (k=='6') lcd_print("Level: High");

        if (sd_ready) sd_write_block(log);
        break;
      }
    }

    while(1);
  }
```

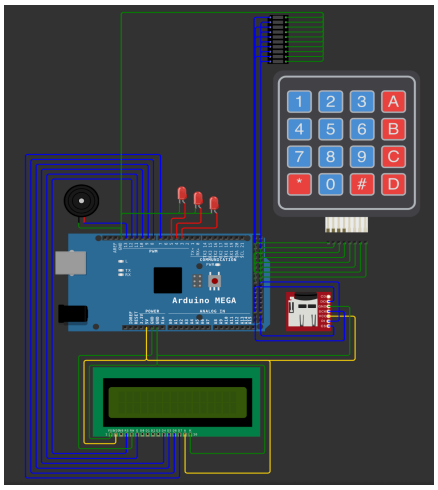
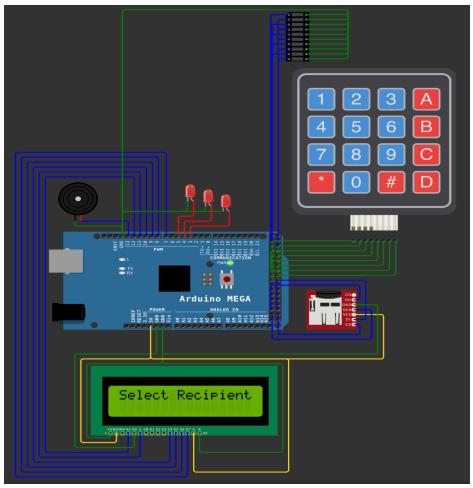
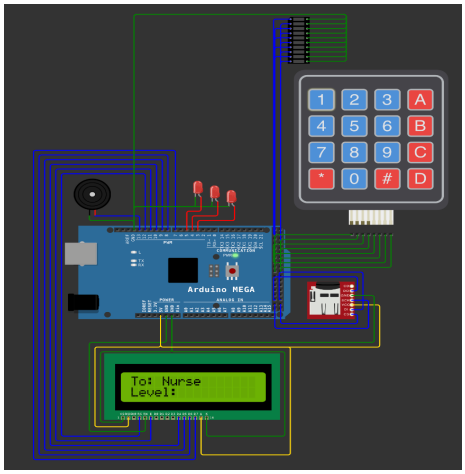
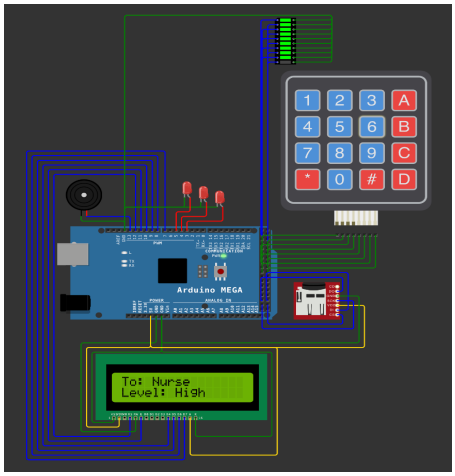
180.2. WOKWI diagram.json File

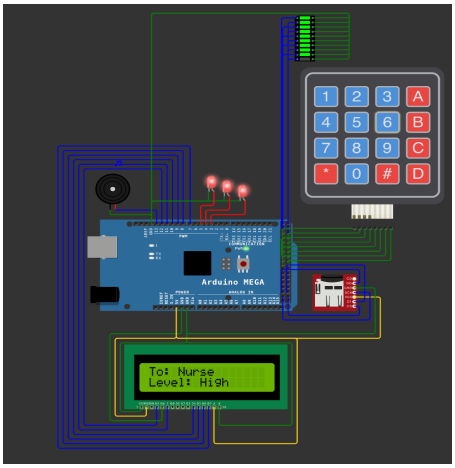
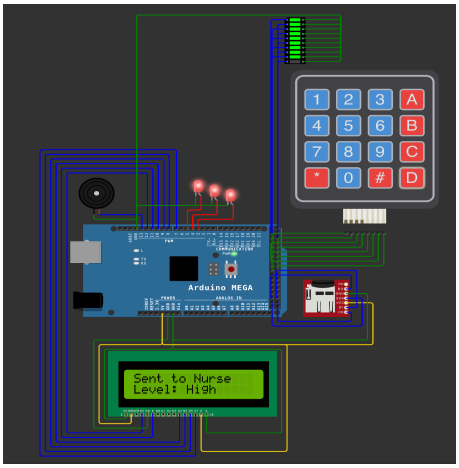
```
1.  {
2.    "version": 1,
3.    "author": "Ecem Gunes",
4.    "editor": "wokwi",
5.    "parts": [
6.      { "type": "wokwi-arduino-mega", "id": "mega", "top": 173.4, "left": -320.4,
    "attrs": {} },
7.      {
8.        "type": "wokwi-buzzer",
9.        "id": "bz1",
10.       "top": 69.6,
11.       "left": -305.4,
12.       "attrs": { "volume": "0.1" }
13.     },
14.     { "type": "wokwi-lcd1602", "id": "lcd1", "top": 435.43, "left": -252.8, "attrs": {}
    },
15.     {
16.       "type": "wokwi-led-bar-graph",
17.       "id": "bargraph1",
18.       "top": -254.4,
19.       "left": 62.4,
20.       "attrs": { "color": "lime" }
21.     },
22.     { "type": "wokwi-microsd-card", "id": "sd1", "top": 288.23, "left": 96.07,
    "attrs": {} },
23.     { "type": "wokwi-membrane-keypad", "id": "keypad1", "top": -146, "left":
    72.8, "attrs": {} },
24.     {
25.       "type": "wokwi-led",
26.       "id": "led1",
27.       "top": 73.2,
28.       "left": -111.4,
29.       "attrs": { "color": "red" }
30.     },
31.     { "type": "wokwi-led", "id": "led2", "top": 92.4, "left": -53.8, "attrs": { "color":
    "red" } },
32.     { "type": "wokwi-led", "id": "led3", "top": 82.8, "left": -82.6, "attrs": { "color":
    "red" } }
33.   ],
34.   "connections": [
35.     [ "bz1:2", "mega:13", "blue", [ "h76.4", "v28.8" ] ],
36.     [ "bz1:1", "mega:GND.1", "green", [ "v9.6", "h28.8" ] ],
37.     [ "lcd1:VDD", "mega:5V", "gold", [ "v19.2", "h-57.5", "v-163.2", "h112.9" ] ],
38.     [ "lcd1:RS", "mega:12", "blue", [ "v38.4", "h-134.1", "v-537.6", "h144" ] ],
39.     [ "lcd1:RW", "mega:GND.2", "green", [ "v28.8", "h-96.1", "v-182.4",
    "h132.25" ] ],
40.     [ "lcd1:VSS", "mega:GND.3", "green", [ "v9.6", "h-38.4", "v-144", "h122.55" ]
    ],
41.     [ "lcd1:E", "mega:11", "blue", [ "v48", "h-163.2", "v-556.8", "h163.2",
    "v124.8" ] ],
```

```
42.    [ "lcd1:D4", "mega:10", "blue", [ "v57.6", "h-220.8", "v-576", "h182.7" ] ],
43.    [ "lcd1:D5", "mega:9", "blue", [ "v67.2", "h-239.9", "v-595.2", "h202.3" ] ],
44.    [ "mega:8", "lcd1:D6", "blue", [ "v-153.6", "h-221.4", "v614.4", "h259" ] ],
45.    [ "lcd1:D7", "mega:7", "blue", [ "v86.4", "h-278.1", "v-633.6", "h244" ] ],
46.    [ "lcd1:K", "mega:GND.3", "green", [ "v38.4", "h134.4", "v-172.8", "h-194.25"
    ] ],
47.    [ "lcd1:A", "mega:5V", "gold", [ "v76.8", "h153.5", "v-220.8", "h-223.1" ] ],
48.    [ "mega:22", "bargraph1:A1", "blue", [ "h-2.2", "v-440.5" ] ],
49.    [ "mega:23", "bargraph1:A2", "blue", [ "v0" ] ],
50.    [ "mega:24", "bargraph1:A3", "blue", [ "v0" ] ],
51.    [ "mega:25", "bargraph1:A4", "blue", [ "v0" ] ],
52.    [ "mega:26", "bargraph1:A5", "blue", [ "v0" ] ],
53.    [ "mega:27", "bargraph1:A6", "blue", [ "v0" ] ],
54.    [ "mega:28", "bargraph1:A7", "blue", [ "v0" ] ],
55.    [ "mega:29", "bargraph1:A8", "blue", [ "v0" ] ],
56.    [ "mega:30", "bargraph1:A9", "blue", [ "v0" ] ],
57.    [ "mega:31", "bargraph1:A10", "blue", [ "v0" ] ],
58.    [ "bargraph1:C1", "mega:GND.1", "green", [ "h67.42", "v-9.6", "h-364.8",
    "v441.6" ] ],
59.    [ "bargraph1:C2", "mega:GND.1", "green", [ "h67.42", "v-19.2", "h-364.6" ] ],
60.    [ "bargraph1:C3", "mega:GND.1", "green", [ "h67.42", "v-28.8", "h-364.6" ] ],
61.    [ "bargraph1:C4", "mega:GND.1", "green", [ "h67.42", "v-38.4", "h-364.6" ] ],
62.    [ "bargraph1:C6", "mega:GND.1", "green", [ "h67.42", "v-57.61", "h-364.6" ]
    ],
63.    [ "bargraph1:C7", "mega:GND.1", "green", [ "h67.42", "v-67.21", "h-364.6" ]
    ],
64.    [ "bargraph1:C8", "mega:GND.1", "green", [ "h67.42", "v-76.81", "h-364.6" ]
    ],
65.    [ "bargraph1:C9", "mega:GND.1", "green", [ "h67.42", "v-86.41", "h-364.6" ]
    ],
66.    [ "bargraph1:C10", "mega:GND.1", "green", [ "h67.42", "v-96.01", "h-364.6" ]
    ],
67.    [ "bargraph1:C5", "mega:GND.1", "green", [ "h67.42", "v-48.01", "h-364.6" ]
    ],
68.    [ "sd1:VCC", "mega:5V", "gold", [ "h48", "v86.54", "h-376.7" ] ],
69.    [ "mega:50", "sd1:DO", "blue", [ "v1", "h-2.2", "v-48", "h144", "v28.69" ] ],
70.    [ "sd1:DI", "mega:51", "blue", [ "h19.2", "v-76.89", "h-144", "v56.6" ] ],
71.    [ "sd1:SCK", "mega:52", "blue", [ "h28.8", "v57.59", "h-161" ] ],
72.    [ "sd1:CS", "mega:53", "blue", [ "h9.6", "v19.26", "h-131.8" ] ],
73.    [ "sd1:GND", "mega:GND.3", "green", [ "h38.4", "v96.11", "h-347.85" ] ],
74.    [ "mega:32", "keypad1:R1", "green", [ "v-46.9", "h132.2" ] ],
75.    [ "mega:33", "keypad1:R2", "green", [ "v-37.3", "h132.2" ] ],
76.    [ "mega:34", "keypad1:R3", "green", [ "h-2.2", "v-37.2", "h153.9" ] ],
77.    [ "mega:35", "keypad1:R4", "green", [ "v-27.6", "h151.2" ] ],
78.    [ "mega:36", "keypad1:C1", "green", [ "v-27.5", "h170.7" ] ],
79.    [ "mega:37", "keypad1:C2", "green", [ "v-17.9", "h170.2" ] ],
80.    [ "mega:38", "keypad1:C3", "green", [ "v-18.05", "h189.95" ] ],
81.    [ "mega:39", "keypad1:C4", "green", [ "v-8.45", "h189.7" ] ],
82.    [ "led1:C", "mega:GND.1", "green", [ "v19.2", "h-105.2" ] ],
83.    [ "led3:C", "mega:GND.1", "green", [ "v9.6", "h-134" ] ],
84.    [ "led2:C", "mega:GND.1", "green", [ "v0", "h-162.8" ] ],
85.    [ "led1:A", "mega:5", "red", [ "v28.8", "h-24.5" ] ],
```

```
86.     [ "led3:A", "mega:4", "red", [ "v28.8", "h-43.8" ] ],
87.     [ "led2:A", "mega:3", "red", [ "v28.8", "h-63.1" ] ]
88.   ],
89.   "dependencies": {}
90. }
```

6. CHAPTER: PROJECT RUNTIME SCREENSHOT IMAGES

	
1.Layout	**2.Initial View**
	
3.Recipient Selection	**4.Emergency Level Selection**

	
5.Led and Buzzer Active	**6.Notification Sent**

Layout: *This view presents the complete schematic of the Med-Alert System within the Wokwi simulation platform. It includes all essential components such as the Arduino Mega, 16x2 LCD, buzzer, keypad, three discrete LEDs, a 10-segment LED bar graph, and a microSD card module.*

Initial View: *Upon powering the system, the LCD displays the prompt "Select Recipient." This screen indicates that the system is in standby mode and waiting for the user to select a recipient by pressing '1' (Nurse), '2' (Doctor), or '3' (Family) on the keypad.*

Recipient Selection: *When the user presses one of the recipient keys, the first row of the LCD updates to display the chosen contact, such as "To: Nurse." The information remains on the screen, and the system becomes ready for the next step selecting the emergency level.*

Emergency Level Selection: *By pressing '4' (Low), '5' (Med), or '6' (High), the user defines the emergency level. This information is shown on the second row of the LCD as "Level: Med" or "Level: High," depending on the selection. The system is now prepared to initiate alerts based on this input.*

LED and Buzzer Active: *Upon urgency selection, two types of visual alerts are activated: the LED bar graph lights up according to the emergency level (3, 6, or 9 segments), and up to 3 separate LEDs turn on. Additionally, if*

"Nurse" was chosen and the urgency level is Medium or High, the buzzer is triggered for 1–2 seconds, providing an immediate auditory alert.

Notification Sent: *After the selection process is complete, the LCD displays the final confirmation message, such as "Sent to Nurse / Level: High." Simultaneously, this information is written to the SD card in raw format, for example:* To:Nurse, Level:High. *This ensures that the emergency event is logged for future reference or analysis.*

7. CHAPTER: CONCLUSIONS AND FUTURE WORK

In conclusion, the Med-Alert Health System shows that an emergency alert platform can be effectively developed using the Arduino Mega and AVR C programming. The system allows users to select a recipient and urgency level through a keypad, displays feedback on an LCD screen, and provides visual and sound alerts with LEDs and a buzzer. It also records the selected data to an SD card for future reference.

The project was tested in the Wokwi simulation environment, and all components were verified to work correctly. Recipient selection, urgency input, and alert signaling were successfully combined into a user-friendly interface. The system is especially useful for elderly people or patients receiving home care, as it is simple to use and reliable in operation.

For future development, wireless communication like Wi-Fi or Bluetooth can be added to send alerts remotely. Health sensors such as heart rate or motion detectors could help trigger alerts automatically without user input. Improving power efficiency would also make the system more suitable for portable or wearable use. Additionally, a mobile app could be developed for caregivers to monitor and track alerts. With these improvements, the Med-Alert system could become a smarter, more connected, and user-centered solution for healthcare support.

,

8. REFERENCES

Microchip Technology Inc. (2017). *AVR1000b: Getting started with writing C-code for AVR® MCUs* (Application Note No. DS90003262B). https://ww1.microchip.com/downloads/en/Appnotes/AVR1000b-Getting-Started-Writing-C-Code-for-AVR-DS90003262B.pdf

EmbeTronicX. (2023). *LCD interfacing with 8051 microcontroller*. https://embetronicx.com/tutorials/microcontrollers/8051/lcd-interfacing-with-8051-microcontroller/

Microchip Technology Inc. (2018). *Using matrix keypad with AVR® devices* (Application Note No. 00003407A). https://ww1.microchip.com/downloads/en/Appnotes/00003407A.pdf

Explore Embedded. (n.d.). *Buzzer with AVR*. https://exploreembedded.com/wiki/Buzzer_with_AVR

Teach Me Micro. (2023). *How to use a LED bar graph with Arduino*. https://www.teachmemicro.com/how-to-use-a-led-bar-graph-with-arduino/

OpenLabPro. (n.d.). *Interfacing microcontrollers with SD card*. https://openlabpro.com/interfacing-microcontrollers-with-sd-card/

Wokwi. (n.d.). *Wokwi documentation*. https://docs.wokwi.com/

Amazon.com.tr. (2025). *Arduino Mega 2560, LCD 16x2, Keypad, Buzzer and SD Card Module Listings*. https://www.amazon.com.tr/

The Best Robotics Enterprise. (2022). *WOKWI – Arduino tutorial* [Video playlist]. YouTube. https://www.youtube.com/playlist?list=PLfdwhnay89Kn3sd6n_kGYg70cWQKG1n6j

Wokwi. (n.d.). *Wokwi Arduino Mega simulation documentation*. https://docs.wokwi.com/parts/wokwi-arduino-mega